

Docker 2048 Game – Internship Practical Report

Project: 2048 HTML/JavaScript Game deployed using Docker, Docker Compose and Nginx

1. Objective

The objective of this practical is to learn Docker fundamentals by containerizing a simple 2048 web game, managing containers, using volumes and networks, deploying multiple services with Docker Compose, and publishing a custom image to Docker Hub.

2. What is Docker?

Docker is a platform for packaging and running applications in lightweight, isolated containers. A container includes the application and its required runtime environment, helping the application run consistently across systems.

3. Docker Architecture

Docker uses a client-server architecture. The Docker CLI sends commands to the Docker Engine. The Engine builds images, creates and runs containers, and manages networks and volumes. Images can be stored and shared through registries such as Docker Hub.

4. Images vs Containers

Images	Containers
Read-only templates used to create containers	Running instances of images
Can be tagged and pushed to Docker Hub	Can be started, stopped, removed and inspected

5. Dockerfile

The Dockerfile defines how the custom 2048 image is built. This project uses nginx:alpine as the base image and copies index.html, style.css and game.js into the Nginx web directory.

6. Docker Volumes

A Docker volume provides storage managed by Docker and can persist data independently from a container lifecycle. The Compose file declares the game-data volume.

7. Docker Networks

A custom bridge network named game-network is used by the Compose services. Containers attached to the same network can communicate using service/container names.

8. Docker Compose

Docker Compose defines the multi-container application in docker-compose.yml. The project includes a custom game service and an Nginx service, along with a volume and network. The application can be started with docker compose up -d --build.

9. Docker Hub

The custom image can be tagged with the user's Docker Hub username and pushed using docker push. The final Docker Hub repository should contain the docker-2048-game:latest image.

10. Practical Commands

```
docker build -t docker-2048-game .
docker run -d --name docker-2048-game -p 8080:80 docker-2048-game
docker compose up -d --build
docker compose ps
docker volume ls
docker network ls
docker images
```

11. Expected Result

After deployment, the 2048 game is available at <http://localhost:8080>. The repository should include the Dockerfile, Docker Compose file, source files, README, screenshots, and this PDF report.

12. Conclusion

This practical demonstrates how Docker can package a web application, create reproducible images, run containers, provide persistent storage, connect services through a custom network, orchestrate multiple containers with Compose, and distribute images through Docker Hub.